GDR Diary – Architettura (arc42)

Oct 27, 2026 · @Andrea

1. Introduzione e obiettivi

GDR Diary è una web app in cui ogni giocatore di ruolo tiene traccia dei propri personaggi e dei loro progressi narrativi sotto forma di diari. Ogni diario ha al massimo 100 pagine (numero scelto in onore dei vecchi libri degli incantesimi dei maghi nei vecchi GdR). Più avanti ogni utente potrà inserire anche le schede dei propri personaggi e altri appunti sotto forma di file.

La versione minima (MVP) ha sei funzioni: registrazione e login, profili personaggio (di tipo giocatore o master), più diari per personaggio con copertina, pagine di testo, caricamento delle immagini, uso comodo da telefono.

Obiettivi di qualità

Sono tre, in ordine di importanza.

1. **Integrità e conservazione dei contenuti.** Chi tiene un diario per anni deve poter contare sulla conservazione di ciò che ha scritto. Oggi lo proteggono il salvataggio automatico con copia sul dispositivo, il lock ottimistico sulle pagine e le transazioni sulle cancellazioni. L'esportazione e i backup sono previsti ma non esistono ancora.
2. **Qualità dell'esperienza di scrittura.** È il punto di forza del prodotto: due pagine alla volta, come un vero diario, e un editor di testo avanzato.
3. **Protezione dei dati personali.** Ogni richiesta controlla che i dati siano dell'utente e le password restano solo in Keycloak. Il GDPR è un requisito: c'è l'informativa privacy e la cancellazione dell'account elimina i dati dal backend e anche l'utente su Keycloak.

2. Vincoli

I vincoli sono pochi: il progetto è di una persona sola, fatto nei tempi liberi e senza scadenze, e quelli davvero esterni sono la privacy e la licenza. Le tecnologie scelte non sono vincoli ma decisioni, e stanno nelle sezioni 4 e 9.

Tipo	Vincolo	Conseguenza
Organizzativo	Una persona, tempi liberi, nessuna scadenza	Si lavora a piccoli passi; la versione minima resta piccola

Tipo	Vincolo	Conseguenza
Privacy	GDPR: informativa e cancellazione dei dati	L'informativa è nell'app; la cancellazione deve coprire anche Keycloak
Licenza	PolyForm Noncommercial 1.0.0 su entrambi i repository	Il codice si legge e si modifica, ma non si usa a fini commerciali
Esterno	Il server di MinIO è archiviato (aprile 2026)	Non si usa; resta solo il client Java verso lo storage S3

I vincoli infrastrutturali (hosting, budget, dominio) non sono ancora definiti e arriveranno in seguito.

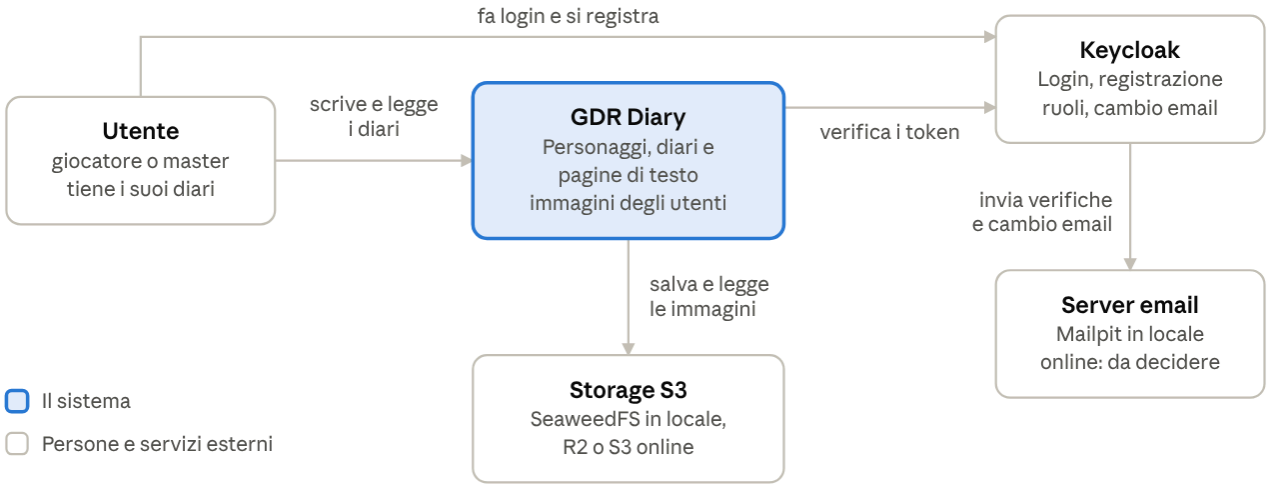
3. Contesto e ambito

GDR Diary ha oggi un solo tipo di utente ed è predisposto per ospitarne un secondo, l'amministratore (admin), che verrà implementato più avanti. Si appoggia a tre servizi esterni:

- Keycloak, per login, registrazione e ruoli (OIDC).
- Storage S3, per i file (oggi le immagini).
- Server email, usato da Keycloak per la verifica dell'indirizzo e il cambio email.

Lo schema seguente riporta la situazione attuale senza tenere conto delle successive implementazioni.

Login, file ed email sono servizi esterni



contesto · 1 utente, 1 sistema, 3 servizi esterni

L'utente usa l'app per i diari e parla direttamente con Keycloak per accedere. L'app verifica i token di Keycloak e salva le immagini sullo storage; le email le manda Keycloak, non l'app.

Dentro l'ambito: personaggi e profili master, diari con copertina, pagine di testo (al massimo 100 per diario), immagini degli utenti, impostazioni di scrittura e aspetto, cancellazione dei dati dell'utente.

Fuori dall'ambito, per ora: tag, ricerca, esportazione del diario, condivisione con altri utenti, campagne e bacheca del master, schede personaggio, immagini generate con intelligenza artificiale e piano premium.

4. Strategia di soluzione

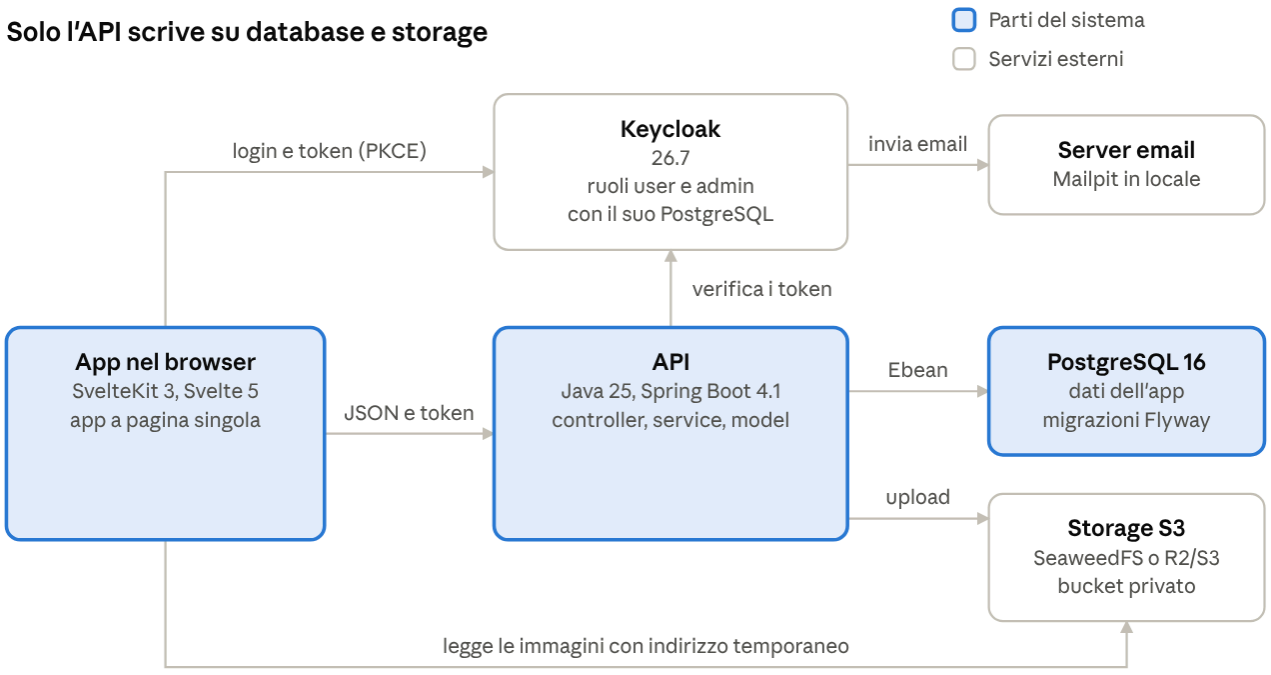
L'architettura è una classica app a tre pezzi (browser, API, database) in cui l'identità e i file sono delegati a servizi specializzati, così il codice da scrivere e da difendere resta poco. Le scelte di fondo, collegate agli obiettivi della sezione 1:

Obiettivo	Come lo ottiene	Dove è spiegata
Integrità e conservazione dei contenuti	Salvataggio automatico con una copia sul dispositivo, controllo dei conflitti tra schede aperte sulla stessa pagina, una transazione per ogni operazione.	Sezioni 6 e 8
Qualità dell'esperienza di scrittura	Vista a due pagine come un vero diario, con lo sfoglio animato; editor avanzato il cui contenuto viene controllato dal backend.	Sezioni 1 e 8
Protezione dei dati personali	Identità e password in Keycloak, controllo che i dati siano dell'utente a ogni richiesta, token solo in memoria nel browser.	Sezioni 6 e 8

Le decisioni principali sono registrate nella sezione 9, con il motivo e i limiti.

5. Vista a blocchi

Il sistema ha tre parti sue (l'app nel browser, l'API e il database) e usa tre servizi esterni; solo l'API scrive sul database e sui file.



container · 3 parti del sistema, 3 servizi esterni

Il browser fa il login su Keycloak e poi chiama l'API con il token. L'API lo verifica, legge e scrive su PostgreSQL e carica i file sullo storage. Le immagini si leggono con un indirizzo temporaneo di 5 minuti, e il browser le legge direttamente dallo storage.

Backend

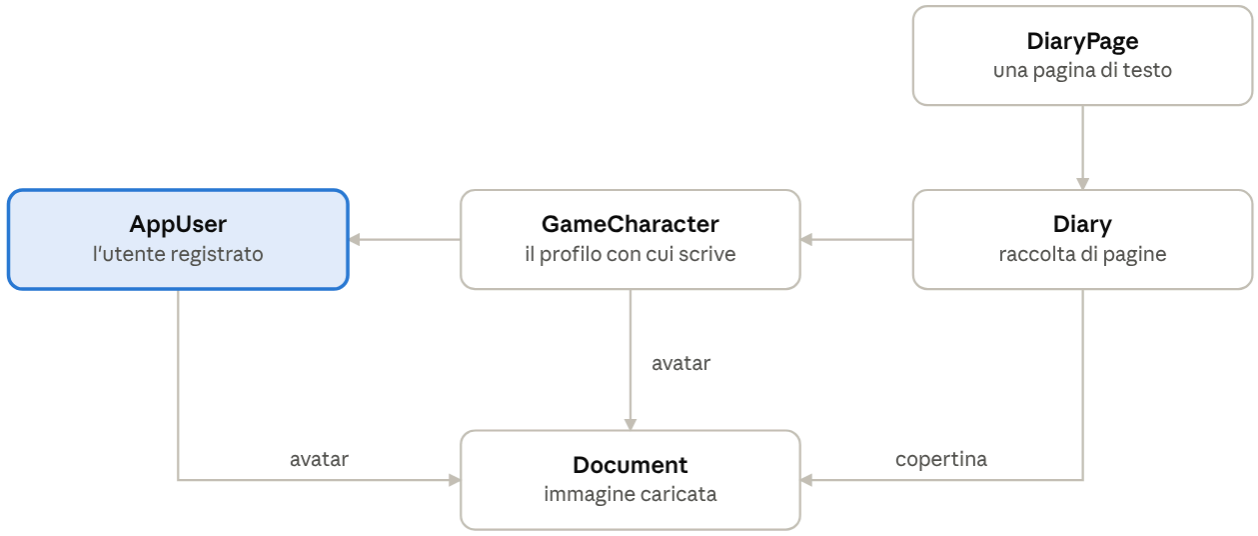
Il backend è un'API Spring Boot che fa solo da resource server: verifica il token di Keycloak, controlla che i dati siano dell'utente, applica le regole dentro transazioni e legge e scrive su PostgreSQL e sullo storage. Il contratto con il frontend è il file OpenAPI.

Frontend

Il frontend è un'app SvelteKit che gira solo nel browser, senza parte server, ed è servita come file statici. Il client dell'API si genera dal file OpenAPI. Il tema di Keycloak sta nello stesso repository.

Modello dei dati

Tutto risale all'utente: personaggio, diario, pagina



La freccia va dal figlio al genitore.
Avatar e copertina sono facoltativi.

entity · 5 tabelle, 6 riferimenti

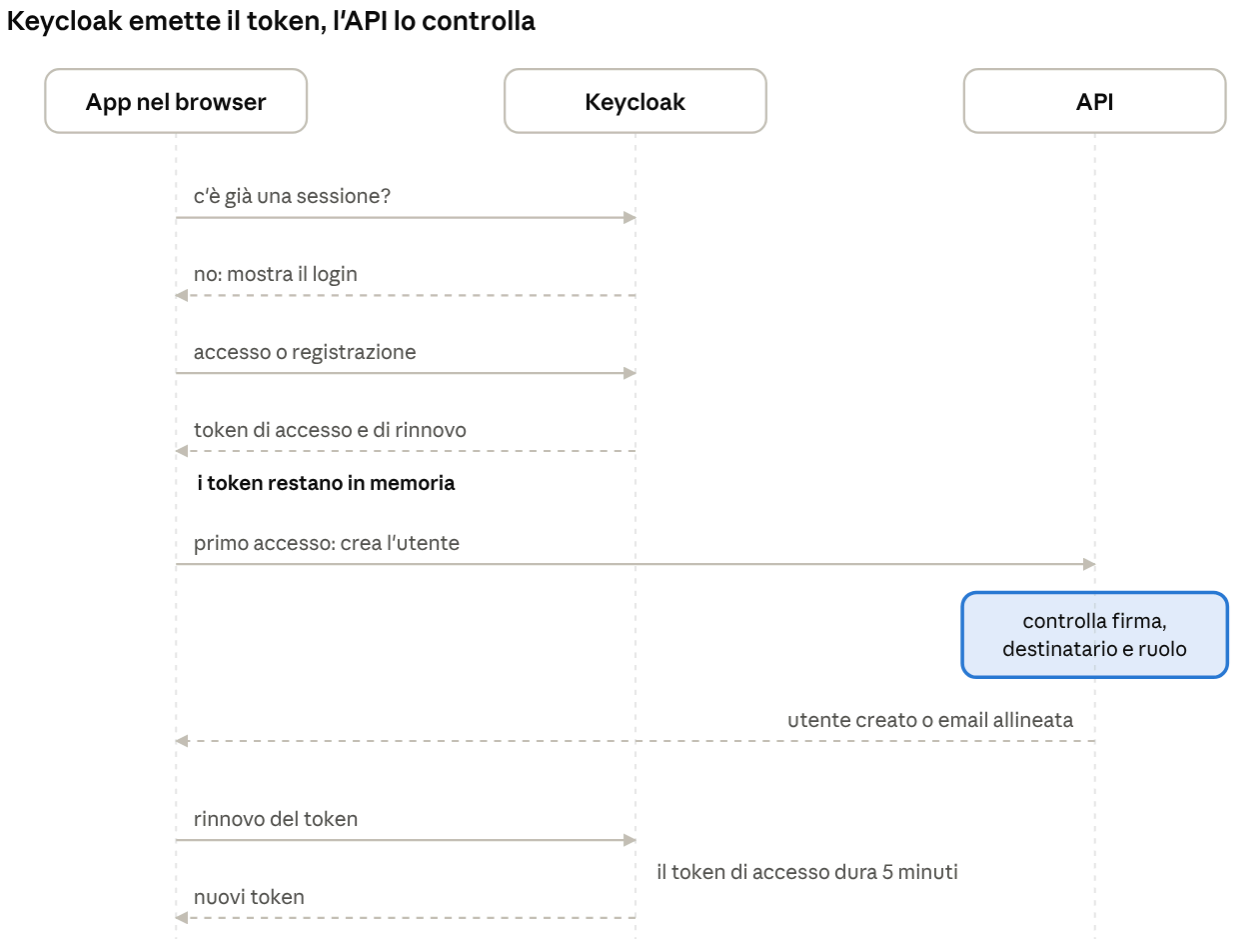
Ogni personaggio appartiene a un utente, ogni diario a un personaggio, ogni pagina a un diario. Le immagini sono `Document` collegati all'avatar dell'utente, all'avatar del personaggio e alla copertina del diario.

Ogni tabella ha un UUID e la colonna `version`. Il numero di pagina, da 1 a 100, è unico per diario e lo assegna il server. Il file di un Document sta nello storage.

6. Vista runtime

Quattro scenari mostrano dove lavora la sicurezza e dove un dato può andare perso: il login con il token, il salvataggio di una pagina con conflitto, il caricamento di un'immagine e la cancellazione dell'account.

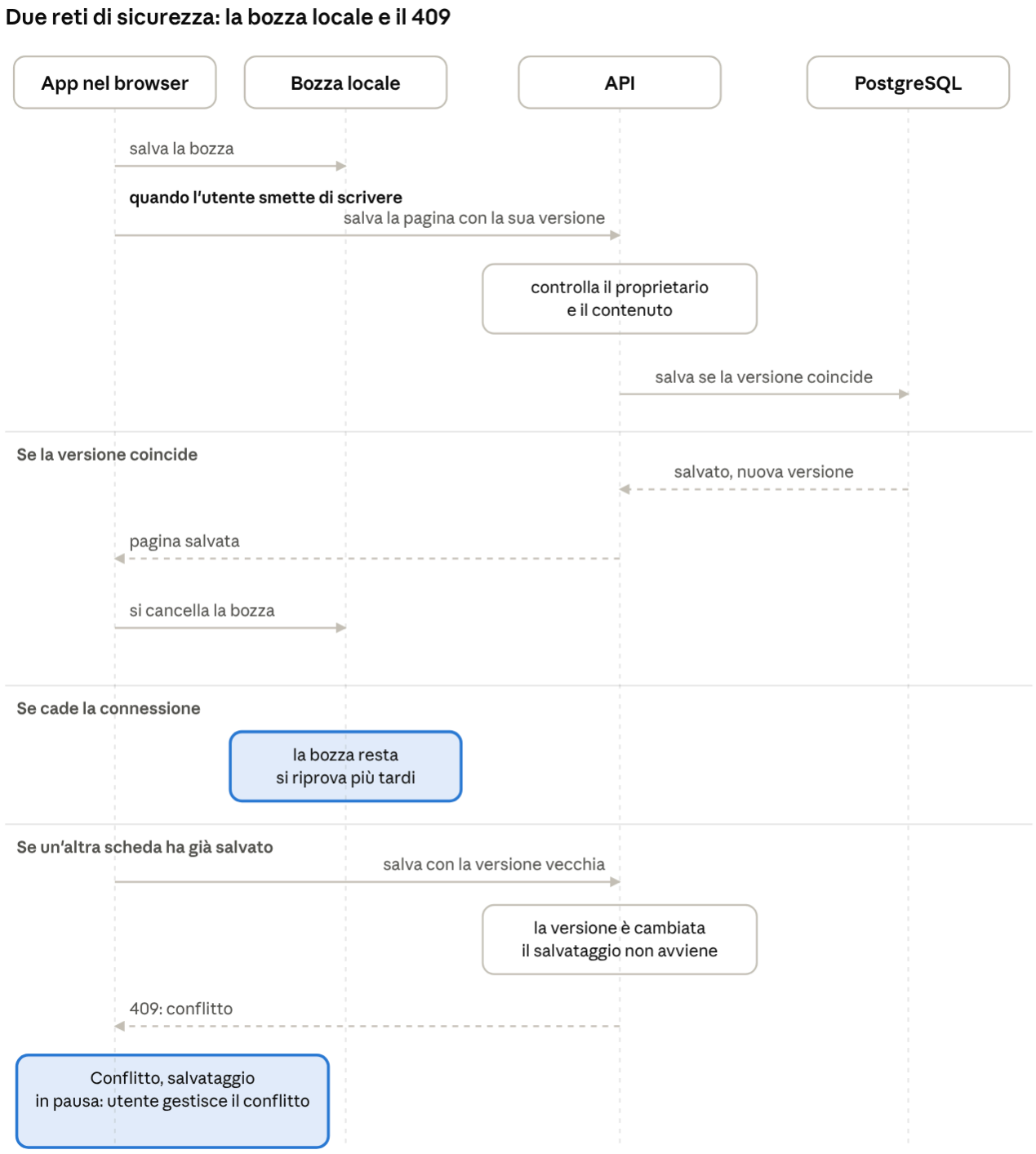
Login e primo accesso



sequenza · login e primo accesso, 3 attori

Se la sessione di Keycloak è ancora valida il login si salta. Al primo accesso l'app crea l'utente nel backend; finché non lo fa, le altre richieste rispondono 404.

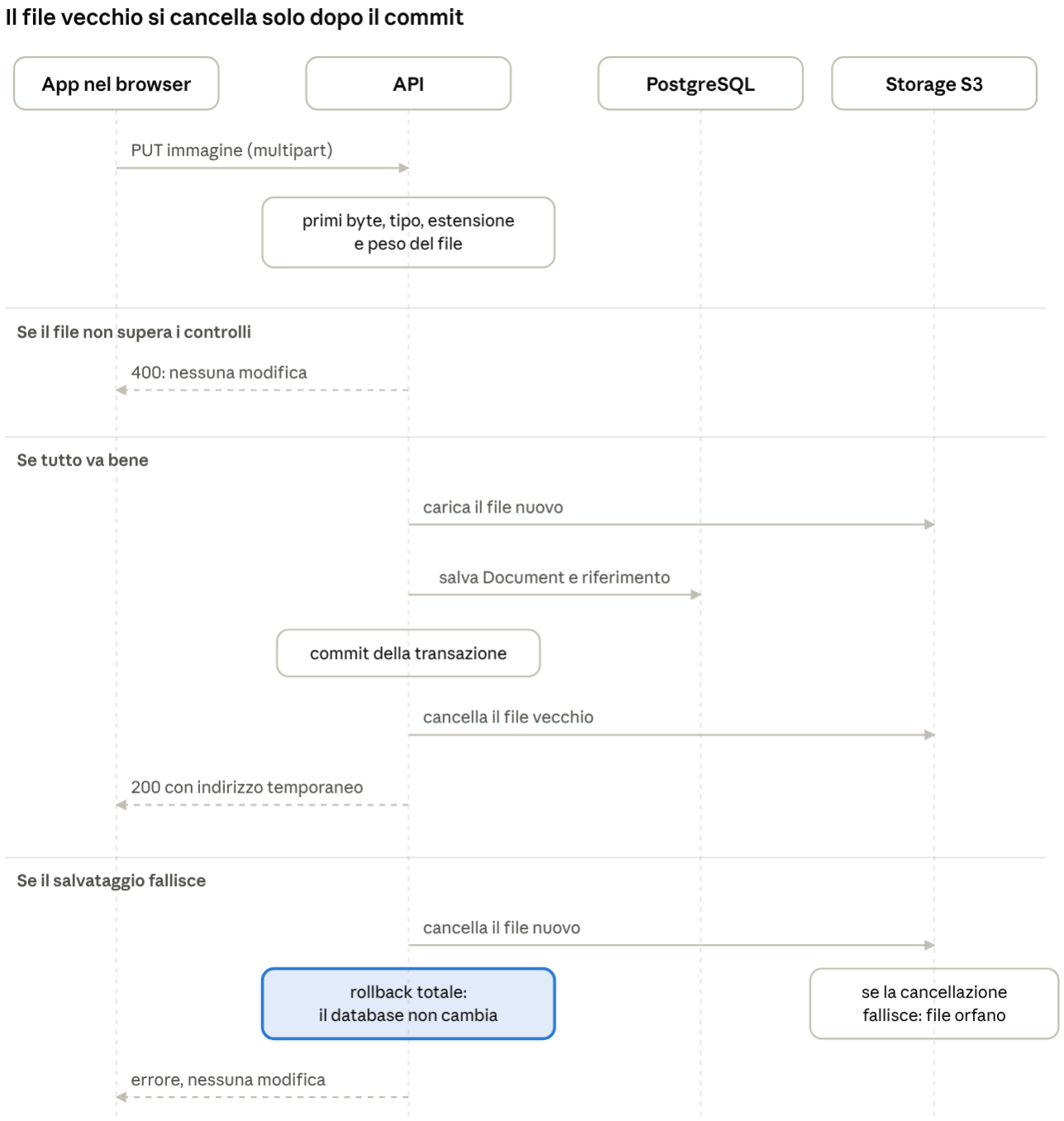
Salvataggio di una pagina



sequenza · salvataggio di una pagina, 3 esiti

Il salvataggio parte da solo quando l'utente smette di scrivere, uno alla volta. Intanto una bozza resta sul dispositivo e si cancella quando il salvataggio riesce. Se la connessione cade si riprova appena torna la rete; se un'altra scheda ha già salvato la stessa pagina, il server risponde 409 e l'utente sceglie quale versione tenere.

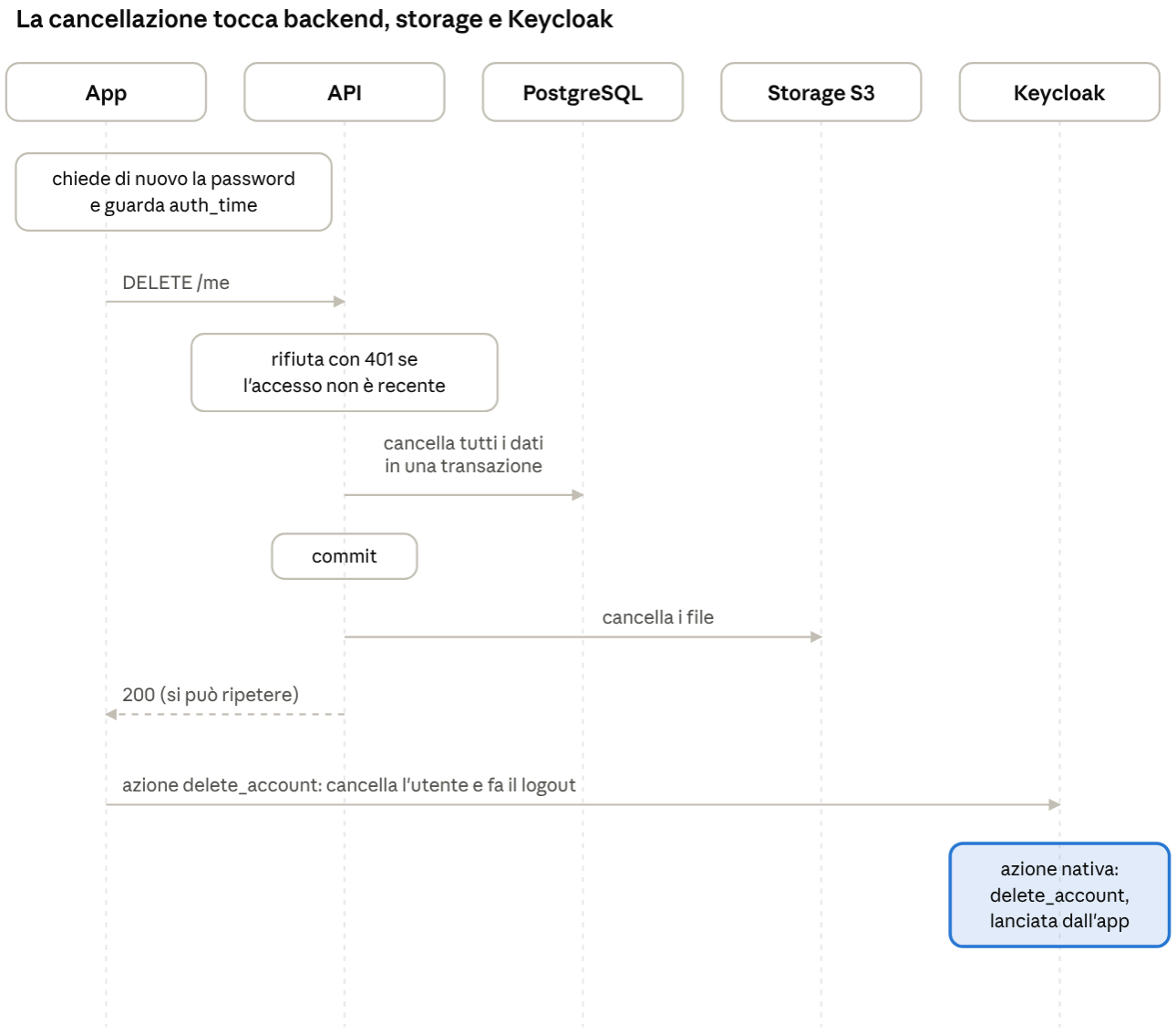
Caricamento di un'immagine



sequenza · caricamento di un'immagine, 3 esiti

Avatar dell'utente, avatar del personaggio e copertina del diario seguono lo stesso percorso: il backend controlla il file, lo salva sullo storage e sostituisce quello precedente. La risposta contiene un indirizzo temporaneo per leggere l'immagine.

Cancellazione dell'account



sequenza · cancellazione dell'account, 5 attori

`DELETE /me` cancella in una sola transazione personaggi, diari, pagine, immagini e `AppUser`, poi i file nello storage; si può ripetere senza danni. Il frontend chiama `DELETE /me` e poi lancia su Keycloak l'azione nativa `delete_account`, che cancella l'utente e chiude la sessione. Se questo secondo passo si interrompe resta solo un utente vuoto su Keycloak, e l'utente può ripetere l'operazione.

Prima di cancellare il frontend chiede di nuovo la password; anche il backend rifiuta la richiesta con 401 se l'accesso non è recente.

7. Vista di distribuzione

Oggi esiste solo l'ambiente locale; la produzione non è stata decisa e qui è descritta solo come elenco di cose da fare.

Ambiente locale

Docker Compose avvia i servizi di supporto, tutti con versione fissata. Backend e frontend girano a parte: il compose non li contiene.

Servizio	Versione	Ruolo	Note
PostgreSQL	16	Database dell'app, schema gestito da Flyway	Un altro database, separato, serve a Keycloak
Keycloak	26.7.0	Login, registrazione, ruoli, cambio email	La configurazione iniziale viene importata solo alla prima creazione
SeaweedFS	4.48	Storage S3 per le immagini	Da completare: manca l'attivazione dell'S3 e la creazione del bucket per le immagini
Mailpit	1.32.0	Riceve le email del realm	Casella di posta di prova, consultabile dal browser
Backend	Java 25, Spring Boot 4.1	API	Accetta richieste solo dal frontend locale
Frontend	SvelteKit 3	App a pagina singola	Il tema grafico di Keycloak sta nel repository del frontend e il compose lo monta

L'estensione di Keycloak per il cambio email va costruita prima dell'avvio. Se cambia la configurazione iniziale, va azzerato il database di Keycloak e ricreato il contenitore, oppure si cambiano le impostazioni dalla console e si riportano a mano nel file di configurazione.

Produzione (work in progress)

Dai documenti del progetto risultano questi punti aperti:

- Ambiente Keycloak di produzione separato da quello di sviluppo, senza accesso diretto con password; da valutare un client riservato.
- Invio email vero al posto di Mailpit.
- Protezione dai tentativi ripetuti e regole sulle password.
- Storage: Cloudflare R2 o S3, cambiano solo indirizzo e chiavi.
- Il backend deve accettare richieste solo dal dominio esatto del frontend.

8. Concetti trasversali

Quattro regole attraversano tutto il sistema, una per sottosezione: la sicurezza, la consistenza dei dati, la validazione di ciò che arriva da fuori e la privacy.

Sicurezza

Le identità stanno in Keycloak. Il backend non emette token e non assegna ruoli: li legge dal token. I ruoli sono due, utente e amministratore; il ruolo utente lo ottiene chi si registra, quello di amministratore ad oggi si dà a mano.

- Per le operazioni sul proprio profilo basta essere autenticati; tutto il resto richiede un ruolo, altrimenti il server risponde 403.
- Il token deve essere destinato al backend (audience): lo scrive Keycloak e lo controlla il backend.
- Ogni query verifica che i dati appartengano all'utente che chiama.
- Nel browser il token resta solo in memoria; il token di rinnovo non si salva mai nel browser.
- Le sessioni sono brevi: il token di accesso dura 5 minuti; la sessione normale scade dopo 30 minuti di inattività (10 ore in totale), con "Ricordami" dopo 15 giorni (30 in totale).

Consistenza dei dati

Ogni operazione su più dati è una sola transazione. Lo storage dei file non è transazionale, quindi il file nuovo si carica dentro la transazione e si cancella se questa fallisce, mentre i file vecchi si cancellano solo dopo che la transazione è andata a buon fine. Se quella cancellazione fallisce restano file orfani: è un limite noto e verrà gestito in futuro.

Le pagine hanno un lock ottimistico: se un'altra scheda ha già salvato, il server risponde 409 e l'utente sceglie quale versione tenere. Il numero di pagina lo assegna il server; le pagine sono consecutive da 1 a 100 e dopo una cancellazione si rinumerano nella stessa transazione.

Validazione del contenuto

Il testo delle pagine è il JSON dell'editor, al massimo 50 KB e profondità 50. Il backend accetta solo ciò che l'editor produce davvero e rifiuta il resto con un errore 400 che indica il punto. Il backend non salva HTML e il frontend non inserisce mai HTML grezzo nelle pagine, quindi non serve un controllo dell'HTML.

Le immagini passano dal backend, che controlla il contenuto vero del file, il tipo dichiarato, l'estensione e il peso. Si accettano solo png, jpeg e webp; i PDF arriveranno con la scheda personaggio. Si leggono con indirizzi temporanei di 5 minuti.

Privacy

L'informativa sulla privacy è nell'app. La cancellazione dell'account elimina i dati dal backend e i file dallo storage in una sola operazione che si può ripetere, e il frontend chiama in successione anche la cancellazione dell'utente su Keycloak. Le password non entrano mai nel database dell'app: restano solo in Keycloak.

9. Decisioni architetturali

Sono le nove decisioni che reggono il sistema oggi, scritte in forma di ADR: decisione, motivo, conseguenze e limiti.

Decisione	Motivo	Conseguenze e limiti
ADR-01 · L'identità sta in Keycloak; il backend è solo resource server	Keycloak offre già pronte le funzioni di login e registrazione basate sugli standard OAuth 2.0 e OpenID Connect (OIDC), evitando così di riscriverle da zero	Il backend dipende da Keycloak: se non risponde all'avvio l'app non parte, e Keycloak va configurato e mantenuto a parte.
ADR-02 · Il frontend è un'app con framework SvelteKit	SvelteKit è un framework molto leggero e di facile mantenibilità	A differenza di altri framework SvelteKit è giovane e può contare su un numero minore di librerie compatibili. Per il nostro scopo è un compromesso accettabile.
ADR-03 · Come ORM (lo strumento che collega il codice al database) si usa Ebean al posto di JPA, con PostgreSQL	Ebean è più semplice di JPA: le relazioni non si caricano in anticipo (si decide nella query cosa leggere) e il caricamento ritardato funziona senza errori.	Community e materiale di studio più piccoli di quelli di Hibernate. Le classi delle entity vanno modificate in fase di compilazione (enhancement), altrimenti non funzionano. Gli strumenti pronti di Spring per JPA non si usano.

Decisione	Motivo	Conseguenze e limiti
ADR-04 · Le immagini passano dal backend e stanno su uno storage S3 compatibile (in locale SeaweedFS, online R2 o S3)	Necessità di controllare il contenuto del file e il formato prima di poterlo accettare dentro lo storage.	Incremento della latenza e della computazione, dovendo fare un giro aggiuntivo. L'indirizzo di lettura vale per chiunque lo abbia, fino alla scadenza dei 5 minuti.
ADR-05 · Salvataggio automatico con copia sul dispositivo, più lock ottimistico sulle pagine	Non perdere il testo se cade la connessione, e non far sovrascrivere in silenzio due schede aperte (evento molto raro).	Il salvataggio anche lato client aumenta le risorse richieste all'utente (per esempio la memoria). Si parla di ordini di grandezza di KB, quindi è un compromesso accettabile.
ADR-06 · Il token resta solo in memoria nel browser; dopo una ricarica vale la sessione di Keycloak	Un token salvato nel browser può essere letto da qualunque script della pagina; in memoria sparisce alla chiusura della scheda.	Dopo una ricarica il token va richiesto di nuovo a Keycloak; per restare collegati a lungo serve "Ricordami".
ADR-07 · Ogni operazione su più dati è una transazione; i file si cancellano solo dopo che la transazione è riuscita	Lo storage dei file non è transazionale, quindi l'ordine conta	Se la cancellazione del file vecchio fallisce resta un file orfano che occupa spazio: è un limite noto.
ADR-08 · Il testo delle pagine è il JSON dell'editor, controllato dal backend	Il backend non salva HTML e il testo resta testo, quindi non serve un controllo dell'HTML per evitare XSS.	Serve una whitelist per evitare che il JSON venga riempito di dati sporchi o corrotti. Anche con la whitelist la garanzia di sicurezza non è alta; tuttavia non sono state trovate alternative valide e più sicure.
ADR-09 · Il cambio email lo fa Keycloak, con un'estensione che evita di bloccare l'accesso	Evita che l'accesso resti bloccato mentre il nuovo indirizzo attende la conferma, e fa valere solo l'ultimo link	L'estensione va mantenuta al cambiare delle versioni di Keycloak.

10. Qualità

Gli scenari seguenti mostrano come il sistema reagisce nei casi che contano per gli obiettivi della sezione 1. Nessun requisito ha numeri (utenti, tempi, disponibilità): sono tra le domande aperte della sezione 11.

Integrità e conservazione dei contenuti

Scenario	Come è coperto	Stato
Cade la connessione mentre si scrive	La bozza resta sul dispositivo e il salvataggio riparte	Implementato
Due schede modificano la stessa pagina	Il secondo salvataggio riceve 409 e l'utente sceglie	Implementato, mai provato con due schede
Una cancellazione fallisce a metà	Si annulla tutta l'operazione	Implementato
L'utente vuole portare via il proprio diario	Esportazione prevista	Non esiste; backup non definiti

Qualità dell'esperienza di scrittura

Scenario	Come è coperto	Stato
Uso da telefono e da tablet	Layout dedicato, controllato da test automatici	Implementato e provato

Protezione dei dati personali

Scenario	Come è coperto	Stato
Un utente chiede i dati di un altro	Ogni richiesta verifica il proprietario dei dati	Implementato
Arriva un token emesso per un altro servizio	Il backend controlla a chi è destinato il token	Provato con Keycloak vero
Si carica un file travestito da immagine	Il backend controlla il contenuto vero del file	Implementato
Una pagina contiene dati non prodotti dall'editor	Il backend la rifiuta con 400	Implementato; provato con 1768 testi generati dall'editor

Sulla manutenibilità: il backend ha test unitari e test di integrazione che girano con PostgreSQL, SeaweedFS e Keycloak veri; il frontend ha un controllo unico che verifica la build, il tema di Keycloak e il layout a più larghezze.

11. Rischi

Questi sono i rischi principali del sistema oggi, con quello che è previsto per ciascuno.

Rischio	Perché	Cosa è previsto
Perdita di dati	Chi scrive per anni deve poter esportare e non perdere il diario	Esportazione e backup: non esistono ancora
Cancellazione incompleta	Se la cancellazione su Keycloak si interrompe dopo `DELETE /me`, resta un utente vuoto con la sua email	Azione nativa `delete_account` lanciata dal frontend dopo `DELETE /me`; l'utente può ripetere l'operazione
Caricamenti	Costi di spazio, moderazione, diritti sulle immagini; niente antivirus	Antivirus rimandato
Sessione con più schede	Se più schede rinnovano il token nello stesso momento, Keycloak può chiudere la sessione	Da provare con due schede aperte

Domande aperte per l'autore

- Numeri attesi: quanti utenti, quanto spazio per utente, quanta disponibilità, ogni quanto un backup.
- Dove gira la produzione e con quale budget.

12. Glossario

Termine	Significato
Personaggio / profilo	Il profilo con cui un utente scrive; ha nome, immagine, campagna facoltativa e colore
Tipo di profilo	Giocatore o master; si sceglie alla creazione e non cambia
Diario	Raccolta di pagine di un personaggio, con copertina; al massimo 100 pagine
Pagina	Unità di testo di un diario, numerata dal server; la vista ne mostra due alla volta
`Document`	Immagine o file caricato, collegato a un utente, un personaggio o un diario
Tiptap	Editor di testo del frontend; salva il contenuto come JSON
Keycloak	Servizio di identità: login, registrazione, ruoli, cambio email
Realm	Spazio di Keycloak che raccoglie utenti, ruoli e applicazioni registrate
Client	Applicazione registrata in Keycloak
Audience	Campo del token che dice a quale servizio è destinato
Resource server	Servizio che accetta i token ma non li emette; è il ruolo del backend
Token di accesso / token di rinnovo	Il primo apre le richieste all'API (5 minuti); il secondo serve a ottenerne uno nuovo
Lock ottimistico	Controllo sulla versione del dato: se è cambiato nel frattempo, il salvataggio fallisce con 409

Termine	Significato
ORM	Strumento che collega il codice al database
Ebean / JPA	Ebean è l'ORM usato; JPA è lo standard Java alternativo
Flyway	Strumento che applica le migrazioni dello schema del database
S3 / SeaweedFS / R2	Protocollo di storage a oggetti e due servizi che lo offrono (locale e online)
OpenAPI	Descrizione dell'API da cui si genera il client del frontend
Mailpit	Server di posta di prova per l'ambiente locale
MVP	Versione minima: le sei funzioni di base della sezione 1
ADR	Architecture Decision Record: una decisione con motivo e conseguenze
GDPR	Regolamento europeo sulla protezione dei dati personali